

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Exercício: introdução ao projeto e à análise de algoritmos](#)

Exercício: introdução ao projeto e à análise de algoritmos?

Fase de avaliação

Fase de configuração	Fase de envio	Fase de avaliação	Fase de cálculo da nota da avaliação	Encerrado
		Fase atual ●		
	Envie seu trabalho Prazo limite dos envios: quinta, 25 ago 2022, 23:59 (1422 dias atrás)	Prazo limite da avaliação: quinta, 1 set 2022, 23:59 (1415 dias atrás)		

Seu envio ▼

Você não enviou seu trabalho ainda

Instruções para avaliação ▼

Respostas:

1. Nesta primeira abordagem vamos incrementando a variável k até obter $k^2 > n$, o que nos permite concluir que $\lfloor n \rfloor = k - 1$.

```
Algoritmo: Raiz(n)
  Entrada: Inteiro não negativo.
  Saída: piso(raiz(n)).

  k = 1
  while k*k <= n do
    k = k + 1
  return k - 1
```

Na abordagem a seguir temos a vantagem de não realizar multiplicações, mas o número de iterações é o mesmo (o valor do contador, ou seja, $\Theta(\sqrt{n})$). A ideia é ir somando os primeiros números ímpares até ultrapassar o valor de n . A justificativa é apresentada abaixo.

Propriedade: a soma S_k dos primeiros k primeiros ímpares positivos vale k^2 .

$$S_k = \sum_{i=1}^k (2 \cdot i - 1) = 2 \cdot \sum_{i=1}^k i - k = 2 \cdot \frac{k(k+1)}{2} - k = k^2.$$

Seja $k = \lfloor \sqrt{n} \rfloor$. Pela definição da função piso, temos que k é o inteiro tal que $k^2 \leq n < (k+1)^2$. Como $(k+1)^2 = k^2 + (2k+1)$, concluímos que n é maior ou igual a k^2 , e estritamente menor que k^2 somado ao próximo ímpar.

```
Algoritmo: Raiz(n)
  Entrada: Inteiro não negativo.
  Saída: piso(raiz(n)).

  soma, prox_impar, cont = 0, 1, 0
  while soma + prox_impar <= n
    soma = soma + prox_impar
    prox_impar = prox_impar + 2
    cont = cont + 1
  return cont
```

2.

```
Algoritmo: Elementos_em_comum(A,B)
  Entrada: Duas listas ordenadas de números A[1..m] e B[1..n].
  Saída: Lista com os elementos em comum nas listas A e B.

  i,j,k = 1,1,1
  while i <= m and j <= n do
    if A[i] < B[j] then i = i+1
    else if A[i] > B[j] then j = j+1
    else
      L[k] = A[i]
      i,j,k = i+1, j+1, k+1
  return L
```

Em cada iteração temos o avanço do i com 3 comparação ($i < m$, $j < n$, e $A[i] < B[j]$), ou o avanço do j com 4 comparações ($i < m$, $j < n$, $A[i] < B[j]$ e $A[i] > B[j]$), ou o avanço de ambos os índices com 4 comparações (quando entra no segundo else).

Portanto, o máximo de comparações ocorre quando não temos elementos em comum nas duas listas, evitando assim que os dois índices avancem com apenas 4 comparações. Note que realizar estes dois avanços em iterações distintas produz 7 comparações.

Além disso, para maximizar o número de comparações avançamos até a posição m na lista A , e até a posição $n + 1$ na lista B (encerrando a busca). Isto ocorre, por exemplo, quando os $m - 1$ primeiros elementos de A são menores que todos os elementos em B , e o último elemento em A é maior que todos os elementos em B .

Concluimos então que o máximo de comparações vale $3(m - 1) + 4n$.

Obs.: Como esta análise é muito detalhada, as respostas podem diferir bastante entre os alunos. Nas aulas seguintes aprenderemos a simplificar a análise.

3. a. $a^3 \cdot a^5 = a^{3+5} = a^8$
b. $3^a \cdot 5^a = (3 \cdot 5)^a = 15^a$
c. Não tem como simplificar.
d. $2^{6 \log_4(n)+7} = 2^{6 \log_4(n)} \cdot 2^7 = (2^{\log_4(n)})^6 \cdot 2^7 = (n^{\log_4(2)})^6 \cdot 2^7 = (n^{1/2})^6 \cdot 2^7 = 2^7 \cdot n^3$
e. $n^{3/\log_2(n)} = n^{3/(\log_n(n)/\log_n(2))} = (n^{\log_n(2)})^3 = (2^{\log_n(n)})^3 = 2^3$
f. $\sum_{i=1}^n (2 \cdot 3^i - 4 \cdot i) = 2 \sum_{i=1}^n 3^i - 4 \sum_{i=1}^n i = 2 \left(\sum_{i=0}^n 3^i - 1 \right) - 4 \sum_{i=1}^n i = 2 \cdot \left(\frac{3^{n+1}-1}{3-1} - 1 \right) - 4 \cdot \frac{n(n+1)}{2} = (3$

Envios atribuídos para avaliação ▾

Você não recebeu nenhum envio para avaliar

×

◀ Vídeo: quantificadores existencial e universal (17min)

Seguir para...

Vídeo: logaritmos e exponenciais (12min) ▶